CMPE 492 Final Project Report
VeriFi: A Reputation-Based Platform for Verifiable Financial Predictions

Arda Saygan
Abdurrahman Arslan
Volkan Bora Seki
Yusuf Anıl Yazıcı

Advisor:
Hüseyin Birkan Yılmaz

Contents

Chapter 1

INTRODUCTION

1.1 Broad Impact

The financial media landscape is plagued by unverifiable claims. Influencers, analysts, and self-proclaimed experts routinely make bold financial predictions on social media with no accountability mechanism. When predictions prove correct they are amplified; when wrong, they are quietly deleted or ignored. This asymmetry erodes public trust and enables misinformation to thrive in financial discourse.

Consider a concrete example. A popular financial influencer on Twitter (X) posts: "Bitcoin will crash 50% by next Friday — sell everything now." Thousands of followers see the post and some act on it. When Bitcoin instead rises sharply, the influencer quietly deletes the tweet and later claims he never made such a prediction. A retail investor who lost money following the advice has no recourse: the tweet is gone, there is no proof it ever existed, and the influencer's public track record remains unblemished. On existing platforms this pattern repeats daily, because there is no technical mechanism to hold predictors accountable after deletion. The influencer's reputation is built entirely on selectively showcasing correct calls while erasing incorrect ones — a survivorship bias that misleads followers at scale.

Now imagine the same scenario on VeriFi. When the influencer posts his prediction, the platform separates the social commentary from the structured financial claim — "Bitcoin, Bearish, 50%, by next Friday" — and cryptographically signs the claim with the influencer's private key. A skeptical follower clicks *Save Proof* and downloads the signed claim to their local device. When the prediction fails and the influencer deletes the post, the social text disappears (respecting the right to erasure), but the mathematical proof persists. The follower uploads the saved proof to VeriFi's public verification page, the

system confirms the cryptographic signature matches the influencer's wallet, and the failed prediction is permanently recorded in the influencer's Truth Score. Deletion does not equal deniability.

VeriFi addresses this problem by building a social media platform where every financial prediction is cryptographically signed at the time of posting. Because the signature is generated from the author's private key and bound to the claim content, a prediction cannot be retroactively modified or plausibly denied — even after the original post is deleted. This property, *non-repudiation*, is the core differentiator of VeriFi compared to existing financial social platforms.

The platform benefits several stakeholder groups. Retail investors gain access to a transparent, tamper-proof track record of financial influencers, enabling more informed decisions about whose predictions to follow. Skilled predictors accumulate a credible, verifiable reputation that serves as a professional credential. Researchers and journalists studying financial misinformation gain a public dataset of signed, timestamped predictions with resolution outcomes.

At a societal level, VeriFi promotes financial literacy by shifting discourse toward verifiable, quantified claims and away from vague, unfalsifiable commentary. By making prediction accuracy publicly auditable, the platform creates an economic incentive for quality analysis and a natural disincentive for low-effort, clickbait content.

1.2 Ethical Considerations

Several ethical dimensions require careful attention in the design and operation of VeriFi.

Not investment advice. VeriFi is a reputation-tracking platform, not a financial advisory service. All content is user-generated and not endorsed by the platform operators. The platform displays a prominent disclaimer that no content constitutes investment advice, in compliance with applicable financial regulations.

Privacy and data protection. VeriFi deliberately avoids collecting personally identifiable information (PII). Users are identified solely by their Ethereum wallet address, a pseudonymous identifier derived from a cryptographic key pair. No names, email addresses, phone numbers, or other PII are stored. Post text is deletable by the user to accommodate GDPR and KVKK (Turkish Personal Data Protection Law) rights to erasure. However, the structured claim payload — the financially verifiable component of a prediction — persists even after post deletion. This design reflects the principle

that while personal commentary may be retracted, quantified public financial claims carry a different epistemic status analogous to a published prediction.

Automated extraction limitations. The claim extraction feature assists users in structuring natural-language posts into verifiable claims. The production system uses a deterministic, rule-based extraction engine rather than an opaque model, and is designed so that the extractor serves as an assistant, not a decision-maker: every extracted claim is presented to the user for explicit confirmation, editing, or rejection before publication. The extractor never publishes claims autonomously. The system also flags when a prediction lacks a measurable target or timeframe, prompting the user to provide those fields rather than inferring them, in order to maintain verifiability.

Sybil resistance. A reputation system is only as trustworthy as its identity layer. By anchoring identity to a cryptographic key pair rather than a platform-controlled account, VeriFi ensures that identity manipulation requires actual cryptographic key control. The challenge-response authentication scheme prevents replay attacks. Planned features such as stake-weighted voting for soft claims further raise the economic cost of Sybil attacks.

Fairness in resolution. Claim resolution is performed automatically by an oracle engine that compares the predicted price movement against historical OHLC (open–high–low–close) market data fetched from multiple independent providers. Every resolution emits an immutable `HardClaimEvent` audit record containing the price data used, so resolution decisions are reproducible and auditable. Resolved claim status cannot revert, and manual resolution by a whitelisted admin address remains available only as a fallback when all data providers fail.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

2.1.1 Motivation and Problem Statement

The financial social media landscape lacks any mechanism to hold prediction-makers accountable after the fact. Influencers, analysts, and self-styled experts publish bold financial claims to audiences of millions, yet face no structural consequences when those predictions prove incorrect. Posts can be deleted, edited, or quietly abandoned without trace, leaving followers with no verifiable record of the claims that shaped their decisions. The asymmetry between making predictions and bearing responsibility for them has created a structural trust deficit in financial discourse online.

A widely documented example illustrates the scale of this problem. In early 2022, Do Kwon, co-founder of Terraform Labs and a prominent figure in cryptocurrency social media, publicly and repeatedly declared on Twitter that the Terra ecosystem and its algorithmic stablecoin UST were fundamentally sound and that critics raising concerns about the mechanism were simply wrong — famously responding to a sceptical economist with "I don't debate the poor on Twitter" [6]. In May 2022, UST lost its dollar peg, triggering a cascade that caused the LUNA token to lose over 99% of its value within 72 hours and erasing an estimated $40 billion in market capitalisation [3]. The U.S. Securities and Exchange Commission subsequently charged Do Kwon and Terraform Labs with defrauding investors, and Do Kwon was sentenced in December 2025 for orchestrating what prosecutors described as a $40 billion fraud [8,9]. In the days immediately following the collapse, promotional content published by thousands of influencers who had recommended LUNA

as a high-yield, low-risk asset — along with dismissive responses directed at earlier critics — disappeared from social media feeds. Without archived evidence, retail investors who had acted on those now-deleted recommendations were left with no verifiable record of the specific claims that had influenced their decisions and no means to establish accountability.

This is not an isolated event. The pattern of making confident financial predictions and deleting or disowning them when outcomes do not materialise recurs continuously across stock market commentary, cryptocurrency promotion, and macroeconomic forecasting. Social media platforms provide no native tooling to preserve a signed record of a published prediction. Posts can be removed without trace, and influencer profiles are routinely curated to surface only successful calls while burying failed ones, a form of survivorship bias that systematically misleads audiences about actual accuracy rates. The result is a financial social media environment that is structurally untrustworthy: followers cannot distinguish between a genuinely skilled analyst and one who simply deleted their way to a clean track record.

VeriFi proposes a technical solution to this problem. By cryptographically signing every financial prediction at the moment of publication and anchoring a public Truth Score to tamper-evident claim records, the platform makes it impossible to rewrite a predictor's history retroactively. Deletion of a post removes the social commentary but preserves the cryptographic proof, so any third party who holds a copy of the signed proof package can independently verify that a specific prediction was made, by a specific account, at a specific time. Financial credibility on VeriFi is earned through demonstrated accuracy rather than selective memory.

2.1.2 System Overview

VeriFi is a web-based social media platform for verifiable financial predictions. The central concept is the *Hard Claim*: a quantified, time-bounded financial prediction with the structured fields `{asset, direction, percentage, until}`. Hard claims are cryptographically signed by their author at publication time. When the deadline passes, each claim is resolved as confirmed or rejected by an automated oracle engine that compares the actual asset price movement (from historical OHLC market data) against the prediction; manual resolution by an administrator exists only as a fallback.

Beyond hard claims, the final system supports *Positions* — suggested trades with entry price, stop-loss, take-profit, and lifetime, tracked against market data through a full lifecycle (`pending` → `active` → `confirmed`/`rejected`/`expired`) — and *Channels*, focused sub-communities with membership roles and posting permissions.

Each user accumulates a *Reputation Score* (`rep`), a global reputation metric earned and lost through a claim-bound prediction market: every hard claim opens a constant-product (CPMM) *Reputation Market* where other users stake a fixed amount of rep on YES or NO. When the claim resolves, winning stakers receive their locked payout. The score is publicly visible, non-editable, and tamper-evident because it is anchored to cryptographically signed claim records and an auditable stake ledger.

Authentication is passwordless and wallet-based. Users either generate a native VeriFi wallet (12-word BIP39 mnemonic [4], secp256k1 key derivation via BIP32 [12], private key stored encrypted in browser `localStorage`) or connect an existing MetaMask wallet. Login uses an EIP-191 personal_sign challenge-response flow [7]: the server issues a single-use nonce, the client signs it with the private key, and the server recovers the signing address for verification. No password is ever stored or transmitted.

The platform is built as a monorepo with a Django REST Framework backend (deployed on Render) and a Vite/React/TypeScript frontend (deployed on Vercel). The database is SQLite in development and PostgreSQL in production.

Delivered (v1, final):

- Wallet-based authentication (native BIP39 wallet, MetaMask, and Privy social login with Google)
- Hard claim creation with mandatory EIP-191 signing, feed, and user profiles
- Claim lifecycle management (`undetermined` → `confirmed` / `rejected`)
- Automated oracle-based claim resolution against multi-provider OHLC market data, with full event logging and price charts
- Positions (suggested trades with entry, stop-loss, take-profit, and lifetime) resolved against market data
- Reputation Market: CPMM-based YES/NO staking on every claim with a daily energy allowance
- Rule-based claim extraction from free-form post text (English and Turkish)
- Cryptographic proof generation, download, and standalone verification page (no login required)

- Social layer: follows, likes, nested comments, notifications, channels, search, and asset catalog (crypto, NASDAQ, BIST, forex, commodities)

Out of scope (v1):

- LLM-based claim extraction in production (researched and benchmarked; production uses the deterministic rule-based engine)
- Domain-specific reputation scores
- Monetization and subscription tiers
- Native mobile applications (the web UI is responsive)
- Blockchain anchoring / on-chain timestamping

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

The project follows a milestone-driven development schedule aligned with the course calendar. Four milestones and a final senior presentation have been defined.

The team consists of four members, each contributing across frontend and backend work. External resource dependencies include financial data APIs (Binance, KuCoin, Kraken, CoinGecko, Yahoo Finance, Twelve Data) for oracle-based resolution, and locally hosted language models (via LM Studio) for the LLM extraction research.

2.2.2 Success Criteria

The project will be considered successful if the following criteria are met by the final presentation:

- A registered user can create a hard claim; the claim is cryptographically signed and stored.
- A visitor can browse the public claim feed without registering.
- An expired claim can be resolved (confirmed or rejected) and the author's Truth Score is updated accordingly.
- Any third party can download a proof file and verify it on the standalone proof verification page without logging in.

Table 2.1: Project milestones and their key deliverables.

Milestone	Date	Deliverables
M1	March 8, 2026	Authentication and identity: native wallet (BIP39/secp256k1), MetaMask integration, challenge-response JWT login, unified login UI
M2	April 5, 2026	Core social and claim pipeline: post composer, live claim extraction, claim card UI, signed proof packages, public social feed
M3	May 17, 2026	Resolution, reputation and verification: oracle integration (CoinGecko, Yahoo Finance), Truth Score, proof verifier page, feed filtering by asset
Final	June 11, 2026	End-to-end demo, UI/UX polish, edge case handling, performance pass, final documentation

- The system correctly rejects tampered or forged proof files.
- The feed page loads within 2 seconds under typical usage conditions.
- No critical security vulnerabilities exist in the authentication or claim submission flow.

2.2.3 Risk Analysis

2.2.4 Team Work

The project team consists of four members. Responsibilities are distributed as follows:

The team holds weekly advisory meetings on Tuesdays (17:30–18:00) with the project advisor. Meeting notes and decisions are recorded in the project GitHub wiki. Rush weekends with concentrated development effort have been scheduled around milestone deadlines.

Table 2.2: Identified risks, likelihood, impact, and mitigation strategies.

Risk	**Likelihood**	**Impact**	**Mitigation**
Oracle API downtime or rate limiting	Medium	High	Exponential backoff retries; fallback to secondary data source; manual admin resolution as last resort
LLM extraction quality insufficient	High	Medium	v1 ships with manual structured entry form; LLM extraction is an enhancement, not a blocker
Truth Score formula disputes	Medium	Medium	Formula publicly documented and fixed at v1 launch; transparent claim records allow independent verification
Browser `localStorage` key loss	Medium	High	Mnemonic displayed and backup strongly encouraged at registration; no server-side key recovery by design
Private key leakage from `localStorage`	Low	Critical	AES-256-GCM encryption with PBKDF2-SHA256 (100,000 iterations); key never transmitted to server
GDPR / KVKK compliance	Low	High	No PII collected; post text deletable; claim payloads treated as public reputation data

Table 2.3: Team member responsibilities.

Member	**Primary Responsibilities**
Arda Saygan	Backend architecture; hard claim data models and API endpoints; project repository management
Abdurrahman Arslan	MetaMask integration; frontend PostCard and claim card UI components; requirements specification
Volkan Bora Seki	AI/LLM infrastructure (Agno + LM Studio); multilingual claim extraction; LLM benchmark design
Yusuf Anıl Yazıcı	Native wallet and key management; cryptographic authentication frontend; midterm report; deployment

Chapter 3

RELATED WORK

The problem of tracking and verifying financial predictions has been approached from several directions in both industry and academia.

Prediction Markets. Prediction markets are exchange-traded platforms where users buy and sell contracts on future events, with prices reflecting aggregate probability estimates [11]. Platforms such as Polymarket and PredictIt allow users to stake real money on event outcomes, providing strong accountability: incorrect predictions result in direct financial loss. While effective for accountability, these systems are regulated as financial instruments in many jurisdictions and do not support the social media dynamics — posts, feeds, followers, profiles — that VeriFi targets. Augur [5] extends the model to a decentralised, blockchain-based architecture, but requires on-chain interaction and cryptocurrency holdings, creating a high barrier for casual users.

Social Trading Platforms. eToro and similar platforms allow users to publish and copy investment portfolios, providing a form of reputation tracking based on actual trade performance. However, these platforms require linking real brokerage accounts, making them inaccessible to users who wish to share predictions without executing trades. StockTwits provides a Twitter-like social layer for financial commentary but lacks any systematic verification or scoring mechanism: predictions and outcomes are not tracked.

Reputation Systems. Reputation systems in online communities, such as Stack Overflow's karma system or academic citation indices, have been studied extensively [2]. A key finding is that reputation scores are only as trustworthy as the underlying identity system and the non-repudiability of the actions they measure. VeriFi applies this insight to financial predictions by anchoring reputation to cryptographic identity and signed claim records.

Cryptographic Non-Repudiation. The use of digital signatures for non-repudiation of electronic documents is well established [1]. Ethereum's EIP-191 personal_sign standard [7] extends this to wallet-based signatures,

which VeriFi uses as the foundation of its proof system. The key contribution of VeriFi is applying this mechanism to social media content — specifically financial predictions — so that deletion or denial of a claim becomes cryptographically impossible for anyone who has saved the signed proof.

LLM-Assisted Structured Extraction. Recent work on large language model-based information extraction has demonstrated strong performance on converting natural language into structured records [10]. VeriFi uses an LLM (via the Agno framework with a locally hosted model) to assist users in extracting structured `{asset, direction, percentage, timeframe}` tuples from free-form text. A critical design constraint, derived from team discussions, is that the LLM must not infer missing fields — especially timeframes — but must prompt the user to supply them explicitly, preserving the verifiability of every published claim.

Chapter 4

METHODOLOGY

VeriFi is built as a monorepo web application with clearly separated frontend and backend layers.

Backend. The backend is a REST API built with Django 6 and Django REST Framework (DRF). Authentication uses `djangorestframework-simplejwt` for JWT issuance and validation, and the `eth-account` library for Ethereum signature verification. The backend is stateless: no server-side sessions are maintained. All authentication state is encoded in the JWT token. The database is SQLite in development and PostgreSQL in production (Render).

Frontend. The frontend is a React 19 + TypeScript single-page application (SPA) built with Vite 7. UI components use Tailwind CSS v4 and the shadcn/ui component library. Cryptographic operations (BIP39 mnemonic generation, BIP32 key derivation, secp256k1 signing, AES-256-GCM key encryption) are performed entirely client-side using `@scure/bip39`, `@scure/bip32`, `@noble/secp256k1`, and `@noble/hashes` — pure TypeScript implementations with no native dependencies. The frontend is deployed on Vercel.

Authentication. The authentication system is passwordless and wallet-based. Two wallet modes are supported: (1) a native VeriFi wallet based on BIP39 mnemonic generation and BIP32/BIP44 key derivation, with the private key stored AES-256-GCM encrypted in browser `localStorage`; and (2) MetaMask via the EIP-1193 browser extension API. Both modes use an identical EIP-191 personal_sign challenge-response flow for login, described in full in Section 6.4.

Claim Signing. Every hard claim is cryptographically signed with the author's secp256k1 private key at creation time. The signature covers a canonical JSON serialisation of the claim payload: `{author_address, asset, direction, percentage, until, server_timestamp}`. This pro-

duces a proof bundle that allows any third party to verify — using only the standard Ethereum public-key infrastructure — that a specific user made a specific prediction at a specific time.

Claim Extraction. The system includes a claim extraction endpoint (`POST /api/posts/extract-claims/`) backed by a deterministic, rule-based extraction engine with no AI dependency at runtime. The engine combines an asset whitelist with alias mapping (e.g. "Bitcoin", "BTC", "bitcoin dolar"), fuzzy matching, and a set of regular-expression patterns for directions, percentage and absolute price targets, and relative or absolute timeframes, in both English and Turkish. As the user types a post, the frontend calls the endpoint with debouncing and displays detected claims live; each extracted claim is presented as a Claim Card for explicit confirmation, editing, or rejection before publication. An LLM-based extractor (Agno framework with locally hosted models) was researched and benchmarked in parallel (Section 7.2); the deterministic engine was chosen for production because it is reproducible, fast, free to operate, and cannot hallucinate fields.

Oracle-Based Resolution. Claim resolution is automated by a backend resolution engine. At claim creation, the reference (entry) price is captured from an intraday quote and stored on the claim. Upon expiry — or on demand for an early hit — the engine fetches historical OHLC candles for the claim window from a fallback chain of independent providers (Binance, KuCoin, and Kraken for cryptocurrency; Yahoo Finance and Twelve Data for equities, forex, and commodities; CoinGecko as an additional crypto fallback) and evaluates whether the asset moved at least `percentage`% in the predicted `direction` at any point between creation and the `until` date. Evaluating the peak (or trough) over the whole window, rather than only the closing price at the deadline, means a prediction that was demonstrably correct mid-window cannot be retroactively falsified by a later reversal. Candles before claim creation are excluded so that pre-entry moves on the creation day cannot confirm a claim. Every resolution writes a `HardClaimEvent` audit record, fetched OHLC rows are cached in the database, resolution is idempotent, and a resolved status cannot revert.

Reputation Market. Each user's Reputation Score (`rep`) is earned and lost through a constant-product market maker (CPMM) prediction market attached to every hard claim — the same mechanism used by Polymarket-style prediction markets [11]. The claim author funds the initial market depth with a creator stake (10–100 rep) on the side of their own claim, and pays a small listing fee that is burned. Other users stake a fixed 10 rep on YES or NO; the CPMM prices each purchase, and the buyer's payout is locked at purchase time. When the oracle resolves the claim, winning shares each pay out 1 rep, the creator additionally receives 5% of the losing side's net

pool, and a 5% fee on every trade is burned to control rep inflation. Markets with too little genuine disagreement (fewer than 3 distinct dissenters or 5 total stakers) refund all stakes, closing the trivial-claim farming loophole. A separate daily *energy* allowance (3 per day, lazily granted) limits how many stakes and claims each user can place, keeping the leaderboard competitive for new users. This design — internally "Model G" — was selected through an extensive simulation study described in Section 7.3.

Chapter 5

REQUIREMENTS SPECIFICATION

5.1 Stakeholders and User Personas

Table 5.1: Stakeholders and their primary interactions with the system.

Persona	**Description**	**Primary Actions**
Predictor	Registered user who makes financial claims	Register, login, create claims, view profile
Visitor	Unauthenticated user browsing the platform	View feed, view profiles, verify proofs
Admin	Whitelisted wallet address (Ethereum)	Manually resolve claim status via API
Proof Verifier	Anyone with a downloaded proof file	Upload proof to the standalone verification page

5.2 Functional Requirements

5.2.1 Authentication

The system provides passwordless, wallet-based login. No email or password is stored on any server. Two flows are supported: native BIP39 wallet and MetaMask.

Table 5.2: Authentication functional requirements.

ID	Requirement
AUTH-01	The system MUST generate a 12-word BIP39 mnemonic in-browser on the client without server involvement.
AUTH-02	The system MUST derive a secp256k1 key pair from the mnemonic using BIP32 path `m/44'/60'/0'/0/0`.
AUTH-03	The Ethereum address MUST be derived as the last 20 bytes of `keccak256(uncompressedPublicKey[1:])`, prefixed with `0x`.
AUTH-04	The user MUST explicitly acknowledge that they have saved their mnemonic before registration proceeds.
AUTH-05	The private key MUST be encrypted with AES-256-GCM using a PBKDF2-SHA256 derived key (100,000 iterations, random salt and IV) before being stored in `localStorage`.
AUTH-06	The server MUST reject duplicate wallet address registrations with HTTP 409.
AUTH-07	On successful registration, the server MUST return a JWT access token.
AUTH-08	The client MUST request a nonce via `GET /api/auth/challenge/?address=<addr>`.
AUTH-09	The server MUST generate a cryptographically random 256-bit nonce and store it with a 5-minute TTL.
AUTH-10	The client MUST sign the nonce using EIP-191 `personal_sign` with the decrypted private key.
AUTH-11	The server MUST recover the signer address from the signature and reject if it does not match the claimed address.
AUTH-12	The nonce MUST be deleted from cache immediately after first verification attempt (single-use).
AUTH-13	On successful login, the server MUST return a JWT with the lowercase wallet address claim and a 7-day lifetime.
AUTH-14	The system MUST support MetaMask (EIP-1193 provider) as an alternative login method using the same backend endpoints.
AUTH-15	When MetaMask is used, VeriFi MUST NOT store or transmit the private key; signing is delegated entirely to MetaMask.
AUTH-16	JWTs MUST be stored in `localStorage["verifi_jwt"]`; wallet address in `localStorage["verifi_address"]`.
AUTH-17	All stateful write endpoints MUST validate the JWT and extract the `address` claim to identify the acting user.
AUTH-18	Logout MUST clear the JWT, address, and encrypted private key from `localStorage`.

5.2.2 Hard Claim Creation

Table 5.3: Hard claim creation functional requirements.

ID	Requirement
CLM-01	A hard claim MUST contain all four fields: `asset`, `direction` (`Bullish`/`Bearish`), `percentage` (float, 0–100), `until` (future date).
CLM-02	The `until` date MUST be strictly in the future at time of submission; the backend MUST enforce this constraint.
CLM-03	The `percentage` field MUST be between 0 and 100; validated on the backend.
CLM-04	A hard claim MUST be linked to the authenticated user's wallet address as `author`.
CLM-05	The initial status of a new hard claim MUST be `undetermined`.
CLM-06	The `asset` field MUST reference a pre-defined `Asset` record (id, name, symbol, description) from the backend.
CLM-07	The `text` field contains the user's free-form natural-language description of their prediction.

5.2.3 Claim Extraction

The endpoint `POST /api/posts/extract-claims/` is served by the deterministic rule-based extraction engine described in the Methodology chapter.

5.2.4 Claim Resolution

5.2.5 Cryptographic Proofs

Every claim and position carries a mandatory EIP-191 signature, enforced server-side; the standalone verification page is publicly accessible.

Table 5.4: Claim extraction functional requirements.

ID	Requirement
CLM-E01	The extraction engine MUST extract hard claim candidates from free-form post text in English and Turkish.
CLM-E02	A valid extracted claim requires all four fields: `asset`, `direction`, `target_value`, `timeframe_iso`.
CLM-E03	If a required field is missing, the system MUST prompt the user to fill it in explicitly — no inference or generic defaults.
CLM-E04	Each extracted claim MUST be presented as a Claim Card for user confirmation, editing, or rejection before posting.
CLM-E05	The extraction response schema MUST be: `{claims: [{asset, direction, target_value, target_unit, timeframe_iso, language}]}`.
CLM-E06	The extraction engine is stateless and deterministic; each call is independent and reproducible.

5.2.6 Reputation Market

5.2.7 Feed

5.2.8 User Profiles

5.3 Non-Functional Requirements

5.3.1 Security

5.3.2 Privacy and GDPR / KVKK

5.3.3 Performance and Reliability

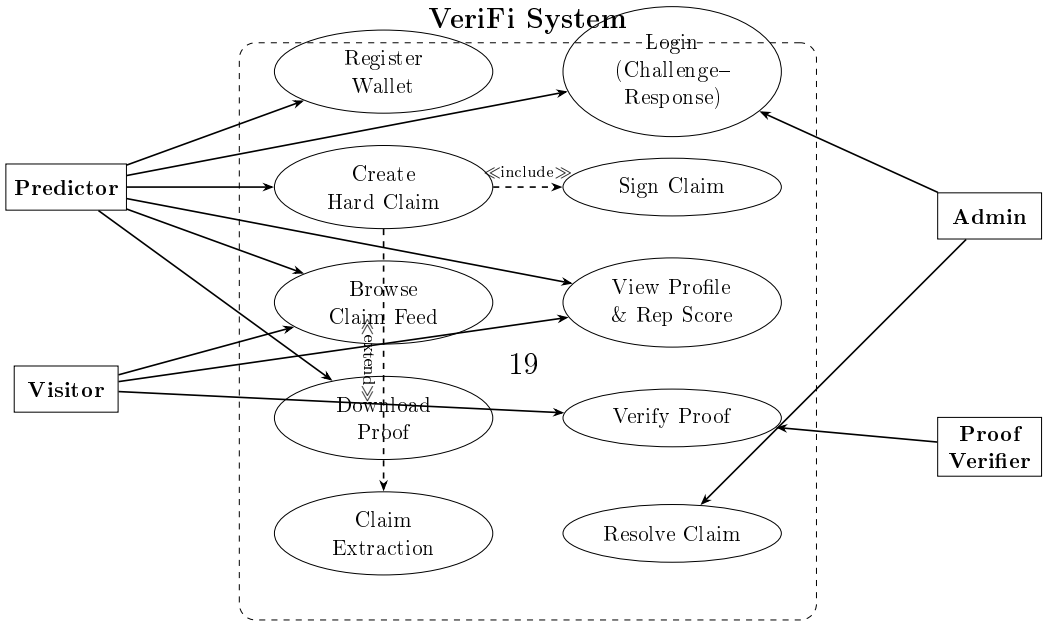

Table 5.5: Claim resolution functional requirements.

ID	Requirement
RES-01	A claim MUST be resolved to either `confirmed` or `rejected` after its `until` date passes.
RES-02	Only wallet addresses listed in `ADMIN_ADDRESSES` (server config) MAY call the claim status update endpoint.
RES-03	The update endpoint MUST accept `{"status": "confirmed"|"undetermined"|"rejected"}` and persist the change.
RES-04	Automated resolution: the system MUST detect expired claims and query the oracle engine to determine the outcome from OHLC data covering the claim window.
RES-05	Oracle selection: Binance / KuCoin / Kraken / CoinGecko for crypto assets; Yahoo Finance / Twelve Data for traditional assets, with per-asset provider symbol mapping.
RES-06	Oracle failures MUST fall through a provider fallback chain; if all providers fail, status remains `undetermined` and manual admin resolution is available.
RES-07	Resolution results MUST be permanent and auditable; a resolved claim status MUST NOT revert.

Table 5.6: Cryptographic proof functional requirements.

ID	Requirement
PRF-01	Every hard claim MUST be signed with the author's secp256k1 private key at creation time.
PRF-02	The proof package MUST contain: `{claim_payload (JSON), signature (hex), wallet_address, server_timestamp}`.
PRF-03	Any authenticated user MUST be able to download a claim's proof as a local JSON file.
PRF-04	A standalone public verification page MUST allow anyone (no login required) to upload a proof JSON and verify its authenticity.
PRF-05	Deleting a post or deactivating an account MUST NOT invalidate a previously generated claim proof.
PRF-06	Proof verification MUST recover the signer address from the signature and compare it to the stored `wallet_address`.

Table 5.7: Reputation market functional requirements.

ID	Requirement
REP-01	Each user MUST have a single global numeric Reputation Score (`rep`), starting at 200.
REP-02	Every hard claim MUST open a CPMM market; the creator funds it with a stake of 10–100 rep on their own side and pays a 2-rep listing fee (burned).
REP-03	A user MAY hold at most one stake per claim, at a fixed 10 rep; a 5% trade fee is burned.
REP-04	Payouts MUST be locked at purchase time (1 rep per winning share); later trades MUST NOT change earlier buyers' payouts.
REP-05	On resolution, the creator MUST additionally receive 5% of the losing side's net pool if their side wins.
REP-06	Markets with fewer than 3 distinct losing-side stakers or fewer than 5 total stakers MUST refund all stakes (trivial-claim protection).
REP-07	Each stake and each claim creation MUST cost 1 energy; users receive 3 energy per UTC day (lazily granted, starting at 4).
REP-08	A claim with status `undetermined` MUST NOT pay out until resolved; all stakes MUST be auditable per user and per claim.

Table 5.8: Feed functional requirements.

ID	Requirement
FEED-01	A public feed MUST display all hard claims sorted by creation time (descending).
FEED-02	Each claim card MUST show: author wallet address, asset, direction, percentage target, deadline, and status badge (`Undetermined` / `Confirmed` / `Rejected`).
FEED-03	The feed MUST be filterable by asset and asset category without page reload.
FEED-04	Feed data MUST be accessible to unauthenticated users.
FEED-05	The feed MUST load in under 2 seconds under normal conditions.

Table 5.9: User profile functional requirements.

ID	Requirement
PRF-P01	Every wallet address MUST have a publicly accessible profile page.
PRF-P02	A profile MUST display: username, wallet address, Reputation Score, and full claim, position, and post history.
PRF-P03	Claim history MUST include a per-claim proof download link.
PRF-P04	Users MUST be able to filter another user's profile by wallet address (search).
PRF-P05	The profile page MUST allow the authenticated owner to reveal their encrypted private key temporarily (60-second auto-hide).
PRF-P06	The revealed private key MUST be displayed partially obfuscated (`first8...last8`) with a copy-to-clipboard option.

Table 5.10: Security non-functional requirements.

Requirement	**Detail**
Private key isolation	Private key MUST never be transmitted to any server. Only the derived public address is known to the server.
Local key encryption	Private key MUST be encrypted with AES-256-GCM + PBKDF2-SHA256 (100,000 iterations) before storage.
Nonce replay prevention	Nonces are single-use; deleted from cache immediately after first verification attempt. TTL: 5 minutes.
JWT security	JWT MUST be signed with `SECRET_KEY` (environment variable in production). Lifetime: 7 days.
CORS	`CORS_ALLOWED_ORIGINS` MUST be restricted to the known frontend origin; wildcard MUST NOT be used in production.
Admin authorization	Only addresses in `ADMIN_ADDRESSES` may call claim resolution endpoints; checked server-side on every request.

Table 5.11: Privacy and data protection non-functional requirements.

Requirement	**Detail**
No PII stored	No email, phone number, real name, or other personally identifiable information is stored.
Post text deletion	Users MUST be able to request deletion of post text (GDPR / KVKK right to erasure).
Claim payload permanence	Signed claim payloads are mathematical accountability data and MAY persist after post text deletion.
Pseudonymous identity	Wallet address is a pseudonymous identifier; users choose how much real-world identity to attach.

Table 5.12: Performance and reliability non-functional requirements.

Requirement	**SLA / Detail**
Feed initial load	< 2 seconds under normal conditions
LLM claim extraction	< 10 seconds per extraction call (when implemented)
Claim resolution job	Must run within 5 minutes of `until` date expiry
Proof verification	< 1 second (client-side computation)
Oracle retries	Exponential backoff on oracle failure; resolution job MUST be idempotent
Claim data durability	Claim records MUST survive post deletions and account deactivations

Chapter 6

DESIGN

6.1 Information Structure

The core data models and their relationships are described below.

Table 6.1: `WalletUser` — registered user identity.

Field	Type	Constraints
`id`	Auto PK	—
`address`	`CharField(42)`	Unique, lowercase Ethereum address (`0x` + 40 hex chars)
`username`	`CharField(30)`	Unique; auto-derived from address if absent
`avatar`	`CloudinaryField`	Optional profile image
`rep`	`FloatField`	Reputation Score; default 200
`energy`	`FloatField`	Daily staking allowance; default 4, +3/day
`created_at`	`DateTimeField`	Auto-set on creation

In addition to the entities above, the final schema includes: `Post` (free-form social post with image attachment), `PostLike`, `PostComment` (nested, with likes), `Channel` and `ChannelMembership` (sub-communities with `owner`/`moderator`/`member` roles and posting permissions), `Follow` (asymmetric user follows), `Notification`, `SavedProof`, `HardClaimEvent` and `PositionEvent` (append-only audit logs of every creation, price check, and resolution), `OHLCData` (cached oracle candles), and `AssetSubscription`

Table 6.2: `Asset` — tradable financial asset with per-provider oracle symbol mapping.

Field	Type	Constraints
`id`	Auto PK	—
`name`	`CharField(100)`	e.g. "Bitcoin"
`symbol`	`CharField(10)`	e.g. "BTC"
`market_type`	`CharField(20)`	`crypto` · `equity` · `forex` · `commodity` · `index`
`provider symbols`	`CharField` ×5	Per-provider identifiers (Binance, KuCoin, Kraken, Twelve Data, CoinGecko/Yahoo) used by the oracle fallback chain
`quote_currency`	`CharField(10)`	e.g. USD, TRY
`usage_count`	`PositiveInteger`	Claim/position reference count; drives search ordering

(observer-pattern registrations linking expiring claims to the resolution engine).

Table 6.3: `HardClaim` — the canonical verifiable financial prediction entity.

Field	**Type**	**Constraints**
`id`	Auto PK	—
`author`	FK → `WalletUser`	Nullable (system claims)
`post`	FK → `Post`	Optional originating post
`channel`	FK → `Channel`	Optional channel scope
`asset`	FK → `Asset`	Required
`direction`	`CharField(20)`	`"Bullish"` or `"Bearish"`
`value_type`	`CharField(20)`	Percentage move vs. absolute price target
`percentage`	`FloatField`	Move magnitude (or target price when `value_type` is `PRICE`)
`until`	`DateField`	Must be > `created_at` (DB constraint)
`reference_price`	`FloatField`	Entry price captured at creation
`signature`	`TextField`	EIP-191 signature over the canonical payload
`claim_payload`	`JSONField`	Canonical signed JSON
`status`	`CharField(12)`	`undetermined` (default) · `confirmed` · `rejected`

Table 6.4: `Position` — suggested trade tracked against market data.

Field	**Type**	**Constraints**
`id`	Auto PK	—
`author`	FK → `WalletUser`	CASCADE delete
`asset`	FK → `Asset`	Required
`direction`	`CharField(10)`	`long` or `short`
`entry_price`	`FloatField`	Price at which the position activates
`stop_loss`, `take_profit`	`FloatField`	Exit thresholds
`lifetime`	`DateTimeField`	Expiry of the position
`exit_price`, `pnl_percentage`	`FloatField`	Set at resolution
`signature`, `position_payload`	`Text`/`JSON`	EIP-191 signed canonical payload
`status`	`CharField(20)`	`pending` · `active` · `missed` · `confirmed` · `rejected` · `expired` · `closed_early`

Table 6.5: `ClaimMarket` and `ClaimStake` — the CPMM reputation market ledger.

Field	Type	Constraints
`hard_claim`	1:1 → `HardClaim`	PK; one market per claim
`y_reserve`, `n_reserve`	`FloatField`	CPMM reserves
`escrow`, `total_burned`	`FloatField`	Rep held / burned in fees
`creator_side`, `creator_stake_rep`	—	Creator's own position (10–100 rep)
`resolved`, `refunded_trivial`	`Boolean`	Lifecycle flags
`market`, `user`	FKs	Unique together; one stake per user per claim
`side`	`CharField(3)`	`YES` or `NO`
`rep_paid_gross/net`	`FloatField`	10 rep gross; net after 5% burn
`shares`, `entry_price`	`FloatField`	Locked payout receipt

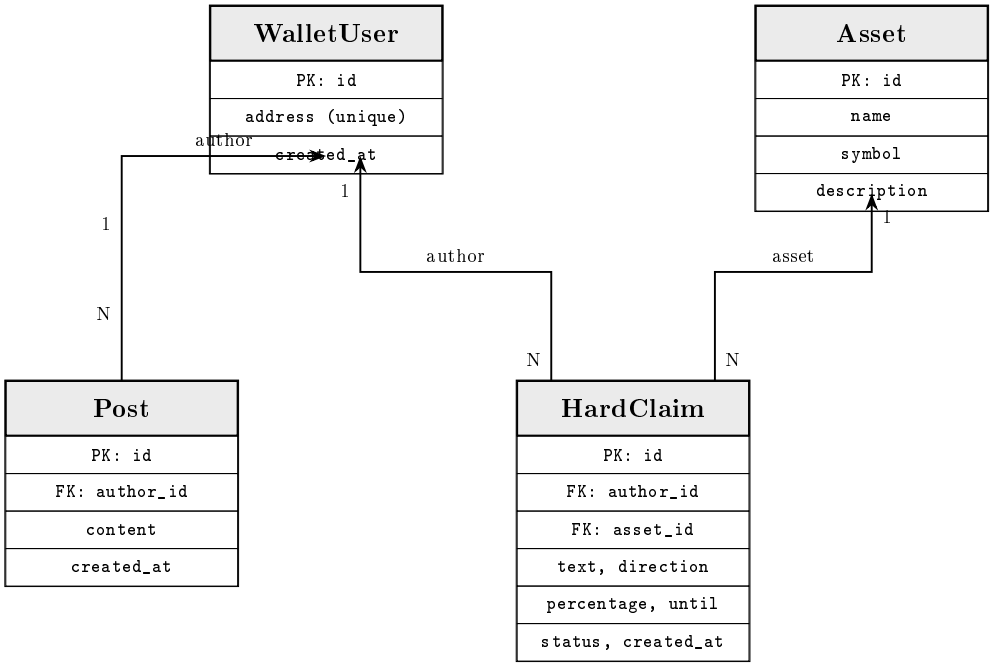

Figure 6.1: Entity-Relationship diagram of the core data models. `HardClaim` references both `WalletUser` (author) and `Asset`. `Post` also references `WalletUser`. Only the core entities are shown; the full schema is listed in the text above.

6.2 Information Flow

Authentication flow (native wallet): The browser generates a mnemonic and derives a secp256k1 key pair entirely client-side. The address is registered with the backend via `POST /api/auth/register/`. On login, the browser requests a nonce from `GET /api/auth/challenge/`, signs it with the private key (EIP-191), and sends the signature to `POST /api/auth/login/`. The backend uses `eth_account.recover_message()` to recover the signing address and issues a JWT on match.

Claim creation flow: An authenticated user composes a post; the rule-based extractor runs live while typing and proposes Claim Cards, or the user fills the structured claim form directly. The client signs the canonical claim payload with the wallet's private key (EIP-191) and sends `POST /api/posts/hard-claims/` with a Bearer JWT header. The backend validates the JWT, recovers the signer address from the signature and rejects mismatches, checks payload consistency and timestamp freshness (± 5 minutes, anti-replay), validates all claim fields, captures the reference price from

an intraday quote, opens the CPMM reputation market with the creator's stake, and stores the record with status `undetermined`.

Claim resolution flow: When a claim's `until` date passes (or an early target hit is detected during a price check), the resolution engine fetches OHLC candles for the claim window through the provider fallback chain, determines whether the target was reached at any point after creation, sets the status to `confirmed` or `rejected`, writes a `HardClaimEvent` audit record with the price evidence, and settles the reputation market: winning stakes receive their locked payout, the creator cut is applied, and trivial markets are refunded. Manual admin resolution via `PATCH /api/posts/hard-claims/{id}/update-status/` remains as a fallback. Expiring claims are connected to the engine through an observer pattern (`AssetSubscription`), so price data fetched once per asset resolves all subscribed claims.

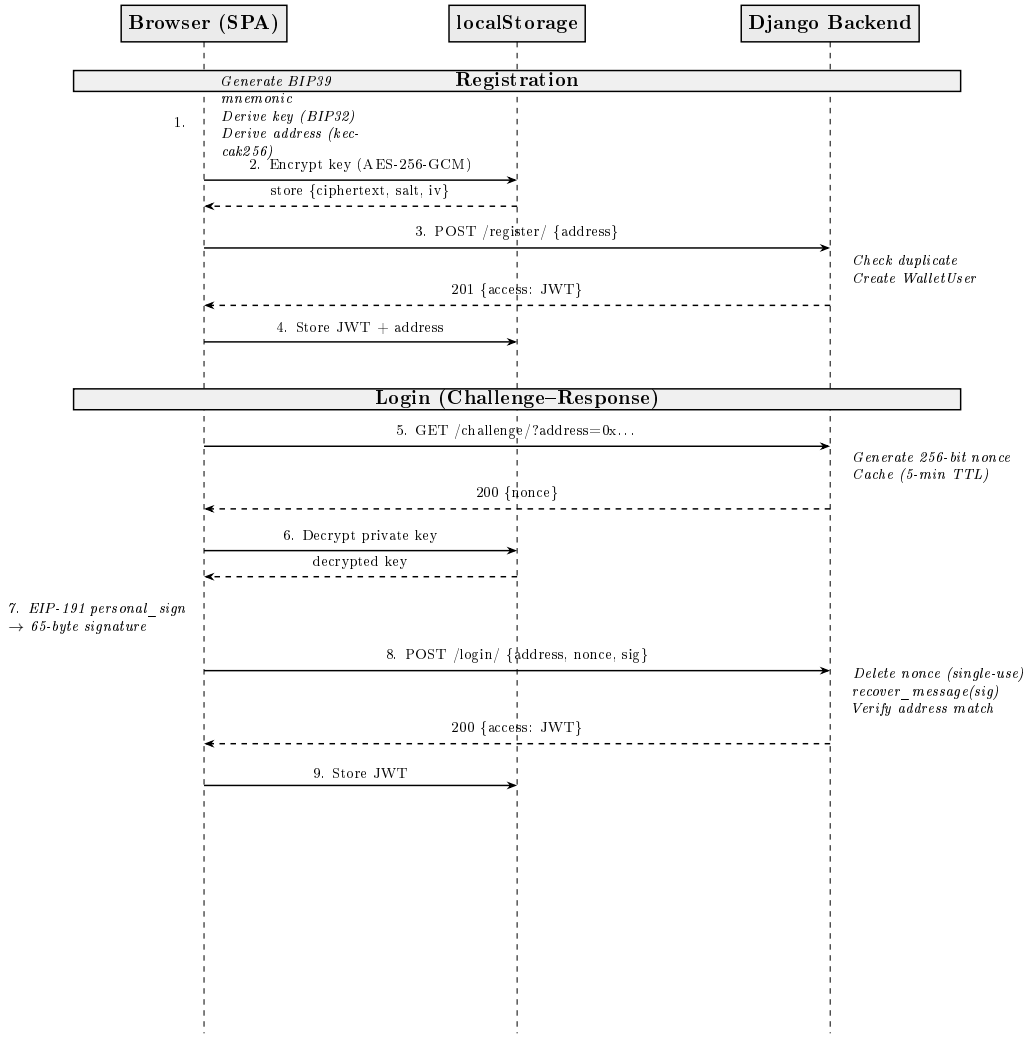

Figure 6.2: Sequence diagram for native wallet registration and challenge–response login.

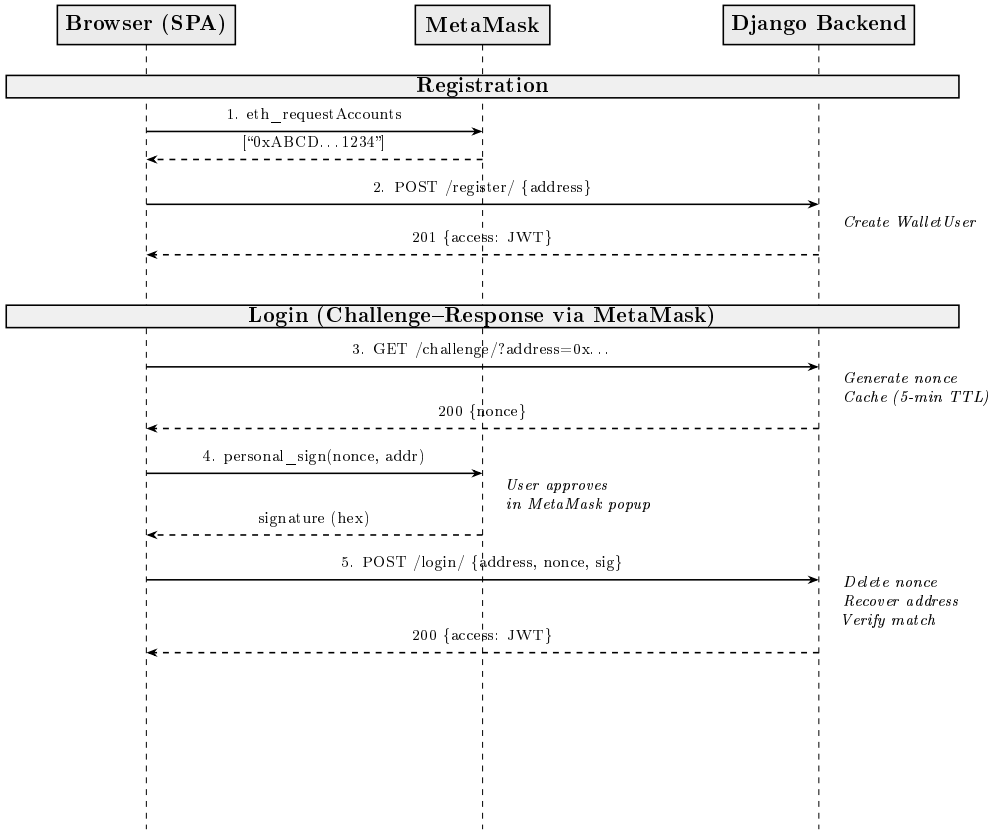

Figure 6.3: Sequence diagram for MetaMask registration and login. The private key never leaves the MetaMask vault.

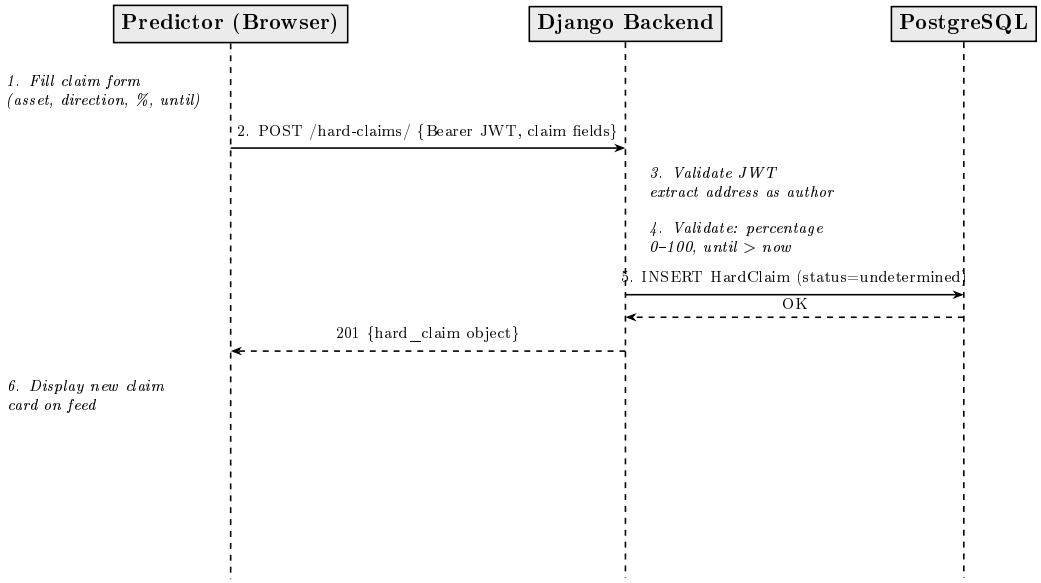

Figure 6.4: Sequence diagram for hard claim creation. The backend validates all fields and stores the claim with initial status `undetermined`.

6.3 System Design

The system follows a strict client-server separation. The frontend is a static SPA deployed on Vercel that communicates with the backend exclusively through the REST API over HTTPS. The backend is stateless: all authenticated state is in the JWT token carried by the client. The private key never leaves the browser.

The REST API is organized into three Django apps:

- `accounts` — authentication, profiles, follows, and energy: `/api/auth/register/`, `/api/auth/challenge/`, `/api/auth/login/`, profile and follow endpoints
- `posts` — content and markets: posts, comments, likes, hard claims, positions, channels, assets and asset search, claim extraction, reputation market stakes, proofs, charts, and search
- `notifications` — per-user notification feed with mark-as-read and delete

API errors follow a uniform envelope `{error: {code, message, fields}}` so the frontend can pin validation messages to the offending form field.

Cross-origin requests from the frontend origin (Vercel domain and `localhost:5173` in development) are handled by `django-cors-headers`.

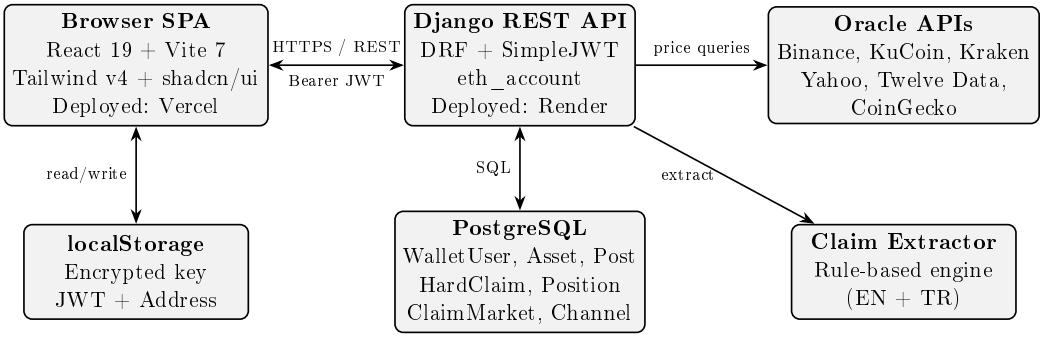

Figure 6.5: High-level system architecture. The frontend communicates with the stateless backend over HTTPS; the private key never leaves the browser.

6.4 Authentication Design

The authentication system is designed around the invariant that the private key is never transmitted to or stored by the server. The server only sees a wallet address (a 20-byte Keccak-256 hash of a public key) and a cryptographic signature over a server-issued nonce.

The native VeriFi wallet uses BIP39 (12-word mnemonic, 128 bits of entropy), BIP32/BIP44 key derivation (path `m/44'/60'/0'/0/0`), and standard Ethereum address derivation. The private key is encrypted with AES-256-GCM using a key derived from the user's passphrase via PBKDF2-SHA256 (100,000 iterations) and stored as a JSON ciphertext object in browser `localStorage`.

Login uses EIP-191 personal_sign: the server generates a random hex nonce (32 bytes, 5-minute TTL, single-use), the client signs it, and the server uses `eth_account.recover_message()` to verify the signature. The following table summarises the security properties of both wallet modes.

Figure 6.6 presents the combined authentication flowchart, showing how the system routes a user through either the native wallet or MetaMask path before converging on the shared challenge–response verification.

Table 6.6: Security properties of native wallet vs. MetaMask authentication.

Property	**Native Wallet**	**MetaMask**
Private key location	`localStorage` (AES-256-GCM encrypted)	MetaMask encrypted vault
Private key transmitted?	Never	Never
Replay attack protection	Single-use nonce (5-min TTL)	Single-use nonce (5-min TTL)
Signature scheme	EIP-191 personal_sign (secp256k1)	EIP-191 personal_sign (secp256k1)
Session mechanism	Stateless JWT (7-day expiry)	Stateless JWT (7-day expiry)

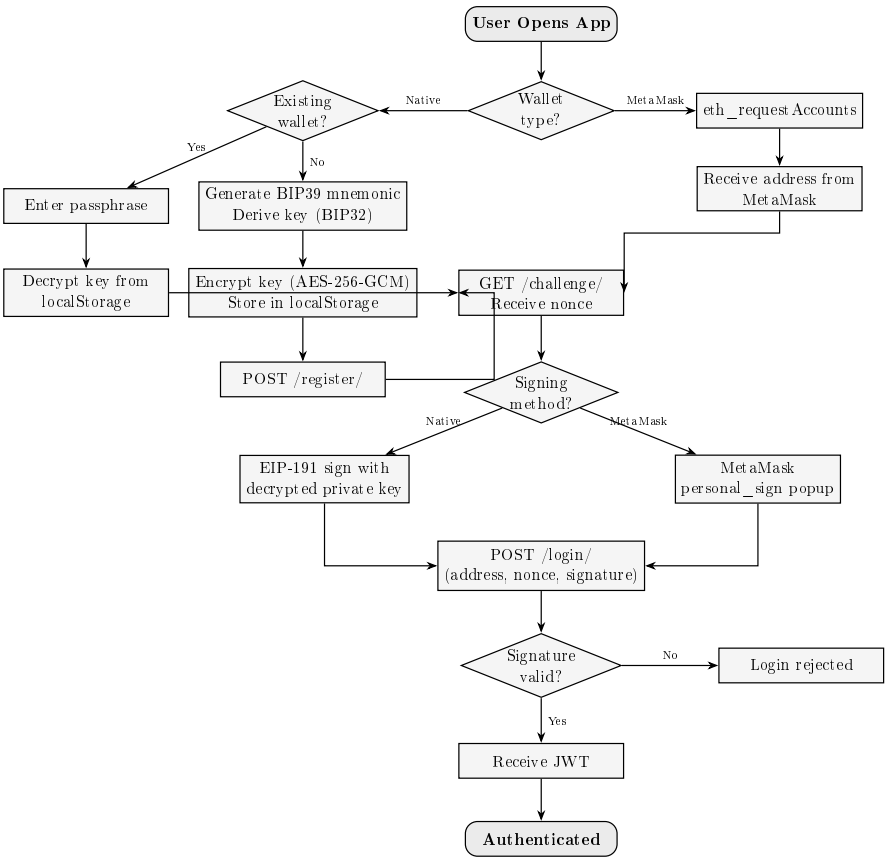

Figure 6.6: Combined authentication flowchart showing native wallet and MetaMask login paths converging at the shared EIP-191 challenge–response verification.

6.5 User Interface Design

The frontend implements the following main views:

- **Login / Registration**: Step-based wallet creation wizard (mnemonic display and confirmation, passphrase setup), MetaMask connect, and Privy social login with Google.
- **Feed Page**: Public stream of posts, claims, and positions in a sidebar layout with a global/following filter, unified search and asset filtering, like/comment/share actions, and smart relative timestamps. A two-phase post composer extracts claims live while typing and supports inline claim and position entry with image attachments.
- **Claim Detail Page**: Expanded claim view with the price chart of the claim window, the reputation market panel (YES/NO prices, stake action, energy meter), the event log, and proof download.
- **Post Detail Page**: Single post with attached claims and positions, nested comments with likes and image replies.
- **Profile Page**: Username, avatar, wallet address (with 60-second timed private key reveal, partially obfuscated), Reputation Score, and tabs for claims, positions, and posts.
- **Proof Verification Page**: Standalone page (no login required) for uploading and verifying a downloaded proof JSON file.
- **Channel Pages**: Channel directory and detail views with membership management and channel-scoped content.
- **Notifications Page**: Per-user notification feed with auto mark-as-read and delete.
- **About Page**: Project information and an interactive Reputation Score simulator.

The design language uses Tailwind CSS v4 utility classes and the shadcn/ui component library for consistent, accessible UI components, with a dark neon theme and a light/dark toggle. The layout is responsive: desktop uses a 1:4:1 sidebar layout, while mobile collapses to a bottom navigation bar. Native browser dialogs are never used; confirmations and password prompts go through dedicated dialog providers, and API validation errors are pinned to the offending input field.

6.6 Design Decisions and Constraints

Table 6.7: Key design decisions and their rationale.

Decision	**Choice**	**Rationale**
Claim target format	Percentage move or absolute price target	Percentage works across asset classes; price targets match how users naturally phrase predictions
Claim extraction	Deterministic rule-based engine	Reproducible, fast, free to run, no hallucinated fields; LLM extractor researched but kept out of production
Claim resolution	Automated oracle over OHLC window	Peak-based evaluation prevents retroactive falsification; admin fallback only on provider failure
Reputation	CPMM market (Model G)	Locked payouts fix late-adoption and copy-trade exploits found in parimutuel scoring (simulation study)
Timestamping	Backend server timestamp	Sufficient for v1; blockchain anchoring is future work
Identity	Wallet address only	Privacy-preserving; no PII stored
GDPR handling	Post text deletable; claim payload permanent	Claim data is accountability data, not personal commentary
Authentication	Passwordless wallet signature	No password storage; cryptographic proof of key ownership
Key storage	`localStorage` (AES-256-GCM encrypted)	Acceptable for v1 target scale; hardware wallet support is future work

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

All core features described in the preceding chapters are implemented and deployed. The implementation effort after the midterm focused on five areas: the resolution engine, the reputation market, the claim extraction engine, positions, and the social layer.

Authentication and identity:

- Native wallet: BIP39 mnemonic generation, secp256k1 key derivation, passphrase-based AES-256-GCM encryption, `localStorage` storage, challenge-response JWT login, delivered as a step-based account creation wizard.
- MetaMask: EIP-1193 `eth_requestAccounts` and `personal_sign` integration using the same backend challenge-response flow.
- Privy social login (Google), synchronized to a wallet identity, alongside the two wallet modes on a unified login page.
- Usernames and avatars (Cloudinary-hosted) on top of the pseudonymous wallet identity; profile changes recorded in an audit changelog.

Claims, signing, and proofs:

- Hard claims supporting both percentage moves and absolute price targets, optionally scoped to a channel or attached to a post.

- Mandatory server-side EIP-191 signature enforcement on every claim and position: signer recovery must match the authenticated wallet, the signed canonical JSON must match the request data, and the payload timestamp must be within ±5 minutes of server time (anti-replay).
- Reference (entry) price captured from an intraday quote at creation and stored with its source URL.
- Proof download and the standalone public verification page; saved proofs tracked per user.

Resolution engine:

- Multi-provider OHLC fetcher with a fallback chain — Binance, KuCoin, Kraken, CoinGecko for crypto; Yahoo Finance and Twelve Data for equities (NASDAQ and Borsa Istanbul), forex, and commodities — with per-asset provider symbol mapping and database caching of fetched candles.
- Window-based evaluation: a claim is confirmed if the target was reached at any candle between creation and deadline; candles before creation are excluded.
- Observer pattern: expiring claims register `AssetSubscription` rows so one price fetch per asset resolves all subscribed claims; every step is logged as a `HardClaimEvent`.
- Position lifecycle tracking: entry triggering, resolution at stop-loss, take-profit, or expiry, PnL computation, and manual early close, with `PositionEvent` audit logs.
- Price chart API serving claim-window OHLC data to the frontend chart in the claim detail view.

Reputation market:

- Model G CPMM implementation: virtual seed, creator-as-trader stake (10–100 rep), 2-rep listing fee, fixed 10-rep trader stakes, 5% burn fee, 5% creator cut, trivial-market refunds — a direct ORM port of the simulator that selected the model.
- Daily energy system: lazy idempotent grant of 3 energy per UTC day; claims and stakes each cost 1 energy.

- Market panel UI with live YES/NO prices, locked-payout display, and an energy meter.

Claim extraction engine:

- Deterministic rule-based extractor (asset whitelist + aliases, fuzzy matching, direction/target/timeframe regex patterns) for English and Turkish, with no runtime AI dependency.
- Debounced live extraction in the post composer: detected claims appear as editable Claim Cards while the user types.

Social layer:

- Posts with image attachments (paste-to-upload), likes, nested comments with likes and image replies, and share actions.
- Asymmetric follows with a global/following feed filter; community content isolated from the main feed.
- Channels with membership workflow (pending/approved/banned), roles (owner/moderator/member), and creator-only posting mode.
- Notifications app with emitters on social events, auto mark-as-read, and deletion.
- Unified search over users, claims, and assets, and a searchable asset catalog covering crypto, NASDAQ, BIST, forex, and commodities with lazy asset resolution.

7.2 AI Claim Extraction Research

In parallel with the deterministic rule-based extractor that ships in production, a specialized AI extraction engine was researched and benchmarked to transform unstructured financial text into verifiable Hard Claims. This section details the framework selection, iterative development, and benchmarking process. The outcome of this research informed the production decision: the deterministic engine matched the LLM's usefulness on the structured patterns users actually type, at zero inference cost and with full reproducibility, while the LLM approach remains the designated path for future, more open-ended extraction (see Conclusion).

7.2.1 Framework Research and Selection

A comprehensive evaluation of Large Language Model (LLM) orchestration frameworks was performed to identify the most suitable architecture for niche financial claim extraction. The following frameworks were researched:

- **MLX & Ollama:** Evaluated for local inference capabilities.
- **LlamaIndex & Langchain:** Analyzed for their broad RAG (Retrieval-Augmented Generation) capabilities.
- **Agno (formerly Phidata):** Selected as the primary framework due to its lightweight agentic architecture. Agno's superior support for structured Pydantic outputs and its specialized focus on agentic workflows aligned perfectly with the project's requirement for high-precision data extraction.

7.2.2 Implementation and Prompt Engineering

The development of the extraction agent followed an evolutionary approach:

(i) **Initial Phase:** A baseline extractor was implemented using standard completion calls to identify simple financial entities.

(ii) **Advanced System Prompting:** The system prompt was iteratively refined into a robust instruction set. This version incorporates few-shot prompting and strict schema enforcement to ensure that extracted claims include all necessary metadata (asset, direction, percentage, and deadline) while minimizing hallucinations in noisy text.

7.2.3 Model Benchmarking and Evaluation

A multi-stage testing protocol was established to determine the optimal model for production.

Phase 1: Exploratory Testing (20-Sample Base Instruction). The following models were tested using a curated set of 20 complex financial sentences: *zai-org/glm-4.6v-flash, mistralai/ministral-8b, deepseek/deepseek-r1-0528-qwen, liquid/lfm2.5-1.2b, ministral-3-8b-instruct, allenai/olmocr-2-7b, qwen2.5-7b-instruct-1m, hermes-3-llama-3.1-8b, gemma-3-27b-it, qwen/qwen3-vl-8b, qwen/qwen3.5-9b, meta-llama-3-8b-instruct,* and *google/gemma-3-4b.*

During this phase, **hermes-3-llama-3.1-8b** exhibited the highest instruction-following capability and structural consistency.

Phase 2: Synthetic Scale Testing. A larger dataset consisting of 1,000 AI-generated samples was used to stress-test the extraction logic. In this scaled environment, **ministral-3-8b-instruct** emerged as the most efficient model, balancing high extraction accuracy with optimized token usage.

7.2.4 The Golden Dataset

To ensure reliability against professional-grade financial discourse, a "Golden Dataset" was curated. This dataset moves beyond synthetic data and draws from high-authority real-world sources, including:

- Corporate annual reports and bank summary statements.
- Central Bank publications and monetary policy reports.
- Financial analyses from institutions such as **Goldman Sachs**.
- Opinions and forecasts from prominent economists and financial columnists.

The objective of this dataset is to fine-tune the extraction agent's ability to isolate "hard claims" from the nuanced and often hedged language of professional financial reporting.

7.3 Reputation Model Simulation Study

The reputation mechanism was the most heavily iterated design decision of the project. Seven candidate models (A–G) were evaluated; the final pick, Model G, was selected through an agent-based simulation study whose code and reports live in the project repository (`report/Rep Score Simulation/`).

The starting point, Model C, was a parimutuel ("split-the-pot") system: everyone who stakes on the winning side splits the entire staked pool. Simulation exposed two structural exploits:

- **Late-adoption penalty.** In a scenario where 5 early skeptics stake NO and 50 latecomers pile onto the eventually-correct YES side, **46%** of the correct latecomers still *lost* rep under Model C, because their slice of the pool was smaller than their stake. Under the CPMM model the figure is **0%**: a correct buyer always profits, just less when buying at a higher price.

- **Copy-trade dilution.** When a skilled predictor's stake is copied by 30 followers, Model C cuts the predictor's profit by **96%** — punishing them for having an audience. Under the CPMM the original predictor's payout is locked at purchase time, so the drop is **0%**; copiers simply buy at a worse price.

The CPMM payout (Model E) fixes both because the number of shares — and therefore the maximum payout — is fixed the moment a user stakes. Two hardening layers were then added: a daily energy allowance (Model F) so that a 30-day leaderboard simulation with 50 users of random skill stays competitive instead of running away, and a creator-funded pool with trivial-market refunds (Model G) so that trivially one-sided claims cannot be farmed for rep. Whale manipulation is impossible by construction: stakes are fixed at 10 rep with one position per user per claim. The production implementation in `posts/rep_market.py` ports the winning simulator verbatim, with the constants listed in the Methodology chapter.

7.4 Testing

Security testing. An XSS vulnerability scan was conducted on 2026-03-17 using XSStrike v3.1.5 targeting both the frontend (`localhost:5173`) and backend (`localhost:8000`) at commit `861894b`. All 28 fuzzer payloads — including `<script>`, `<svg>`, and event handler injections (`onload`, `onclick`) — were blocked. No XSS vulnerabilities were detected. React's automatic content escaping (no `dangerouslySetInnerHTML` usage) and Django REST Framework's JSON-only response format both contribute to this result.

Automated test suite. The backend ships with a structured automated test suite of over 170 test cases run with Django's test runner, covering: claim extraction (English and Turkish patterns, asset aliasing, timeframe parsing, near-miss rejection), the resolution engine (window evaluation, early hits, provider fallback, idempotence), the OHLC fetcher, position lifecycle resolution (entry triggering, stop-loss/take-profit ordering, expiry, PnL), the reputation market (CPMM pricing, locked payouts, burn accounting, creator cut, trivial refunds, energy costs), the observer pattern, API views, and the accounts and notifications apps. The suite runs on every change and gates merges to the development branch.

Extraction benchmark. The claim extraction benchmark dataset contains examples in English and Turkish across four categories: (1) single-claim factual predictions, (2) multi-claim sentences, (3) combined quantitative predictions, and (4) noise / near-miss inputs that should not produce claims

(e.g., intent statements, non-quantified commentary). It was used both to rank the candidate LLMs (Section 7.2) and as the regression suite for the production rule-based extractor.

Simulation testing. The reputation market was validated economically before implementation through the simulation study of Section 7.3, including a worst-case adversarial analysis and an inflation analysis (rep minted vs. burned per user) that set the final burn-fee and seed constants.

Manual testing. All user flows — both wallet authentication modes, social login, post and claim composition with live extraction, staking, resolution, proof download and verification, channels, notifications, and search — were tested manually against the production deployment, including responsive/mobile layouts.

7.5 Deployment

The platform is live in production with the following architecture:

- **Frontend**: Vercel, building from the `frontend/` directory with `pnpm build`. Configuration in `frontend/vercel.json`; SPA routing handled via `rewrites`.
- **Backend**: Render, running `gunicorn` as the WSGI server. Configuration in `backend/render.yaml`. Environment variables (`SECRET_KEY`, `DATABASE_URL`,
  `ADMIN_ADDRESSES`, CORS origins, oracle API keys, Cloudinary credentials) are managed through Render's dashboard and never committed to the repository.
- **Database**: SQLite in development; managed PostgreSQL (Neon) in production, with separate development and production databases.
- **Media**: User avatars and post images are stored on Cloudinary.

Local development requires only two commands: `uv run python manage.py runserver` for the backend (dependencies managed by `uv`) and `pnpm dev` for the frontend. Database seeding commands (`seed_assets`, `seed_feed`, `seed_prod`) populate the asset catalog and demo content, including claims with valid signatures and reputation markets. Build and run instructions are maintained in the repository `README` files.

Chapter 8

RESULTS

All success criteria defined in Chapter 2 are met. The platform is live in production on Vercel (frontend) and Render (backend) with a managed PostgreSQL database, and the full prediction lifecycle works end-to-end: a registered user creates a signed hard claim, the claim opens a reputation market that other users stake on, the oracle resolves the claim against multi-provider market data at (or before) the deadline, the market settles and reputation balances update, and any third party can download the proof and verify the signature on the standalone verification page without logging in. Tampered or forged proof files are correctly rejected, and the server refuses to store any claim whose signature does not recover to its author.

All four milestones were delivered. Milestone 1 (March 8) shipped the complete wallet authentication stack on schedule. Milestone 2 (April 5) shipped the post composer with live claim extraction, claim cards, signed proof packages, and the public social feed. Milestone 3 (May 17) shipped the oracle resolution engine, the reputation market, the proof verifier page, and feed filtering by asset — with the reputation mechanism upgraded from the originally planned weighted formula to the simulation-validated CPMM market. The final stretch delivered positions, channels, notifications, search, the asset catalog, social login, and a full UI polish pass.

Two design outcomes are worth highlighting as results in their own right. First, the simulation study quantified concrete failure modes of the intuitive parimutuel reputation design — 46% of correct-but-late users losing reputation, and a 96% payout collapse for predictors with followers — and demonstrated that the deployed CPMM design eliminates both (0% in each case) while burning fees faster than it mints virtual seed, keeping reputation inflation controlled. Second, the claim extraction comparison showed that for the structured prediction language of the target domain, a deterministic rule-based engine is a competitive, zero-cost, fully reproducible alterna-

tive to LLM extraction; among the thirteen benchmarked LLMs, *hermes-3-llama-3.1-8b* led on instruction following and *ministral-3-8b-instruct* on scaled accuracy-per-token, providing a measured baseline for future LLM integration.

On the security side, the XSS scan found no vulnerabilities, the authentication design (single-use nonces, signature recovery, JWT) withstood end-to-end verification, and the signature enforcement layer guarantees that every claim and position in the database is attributable to a private-key holder — the property the platform exists to provide.

Chapter 9

CONCLUSION

VeriFi brings cryptographic accountability to financial social media — a domain dominated by unverifiable claims and selective recall. By anchoring every prediction to a signed proof tied to a cryptographic wallet identity, the platform makes financial predictions permanently attributable and verifiable, even after the original post is deleted.

The delivered system realises this vision end-to-end. A user authenticates with a native BIP39 wallet, MetaMask, or social login — implementing the full BIP39/BIP32/secp256k1/EIP-191 stack client-side with AES-256-GCM key protection while remaining fully compatible with the Ethereum ecosystem. Every claim and suggested position is signed at creation and rejected by the server if the signature does not recover to its author. An automated oracle resolves claims against OHLC data from six independent market-data providers, logging the evidence for every decision. Reputation is earned through a constant-product prediction market whose design was selected by simulation specifically to reward early, correct, contrarian predictions and to resist late-adoption penalties, copy-trade dilution, trivial-claim farming, and whale manipulation. Around this verifiable core, a complete social platform — feed, follows, comments, channels, notifications, and search over a multi-market asset catalog — makes the accountability mechanism something people would actually use.

Beyond the product itself, the project produced two transferable findings: a quantified comparison of reputation payout mechanisms showing why parimutuel scoring fails socially-networked prediction settings, and evidence that deterministic extraction is a viable production alternative to LLMs for structured financial claim parsing.

Future work follows three threads: training a domain-specific finance LLM for the extraction component, so claim parsing handles the hedged language of professional financial commentary; blockchain anchoring of claim

proofs for trustless timestamping independent of the platform; and domain-specific reputation scores alongside the global one. The architecture was designed so each of these slots in without disturbing the verifiable core.

Bibliography

[1] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ECDSA). *International Journal of Information Security*, 1(1):36–63, 2001.

[2] Audun Jøsang, Roslan Ismail, and Colin Boyd. A survey of trust and reputation systems for online service provision. *Decision Support Systems*, 43(2):618–644, 2007.

[3] Jiageng Liu, Igor Makarov, and Antoinette Schoar. Anatomy of a run: The terra luna crash. Working Paper 31160, National Bureau of Economic Research, 2023. Available at `https://corpgov.law.harvard.edu/2023/05/22/anatomy-of-a-run-the-terra-luna-crash/`.

[4] Marek Palatinus, Pavol Rusnak, Aaron Voisine, and Sean Bowe. BIP-39: Mnemonic code for generating deterministic keys. Bitcoin Improvement Proposals, 2013. Available at `https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki`.

[5] Jack Peterson, Joseph Krug, Micah Zoltu, Austin Williams, and Stephanie Alexander. Augur: a decentralized oracle and prediction market platform. Technical report, Forecast Foundation, 2019. Version 2.0. Available at `https://github.com/AugurProject/whitepaper`.

[6] Marco Quiroz-Gutierrez. Who is do kwon, the 'lunatic' who created a $60 billion cryptocurrency that collapsed in days? *Fortune*, May 2022. Available at `https://fortune.com/2022/05/18/who-is-do-kwon-luna-ust-crypto-founder-rise-and-fall/`.

[7] Geth Team. EIP-191: Signed data standard. Ethereum Improvement Proposals, 2016. Available at `https://eips.ethereum.org/EIPS/eip-191`.

[8] U.S. Department of Justice. Crypto-enabled fraudster sentenced for orchestrating $40 billion fraud. Press Release, U.S. Attorney's Office,

Southern District of New York, December 2025. Available at https://www.justice.gov/usao-sdny/pr/crypto-enabled-fraudster-sentenced-orchestrating-40-billion-fraud.

[9] U.S. Securities and Exchange Commission. SEC charges terraform and CEO do kwon with defrauding investors in crypto schemes. Press Release No. 2023-32, February 2023. Available at https://www.sec.gov/newsroom/press-releases/2023-32.

[10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[11] Justin Wolfers and Eric Zitzewitz. Prediction markets. *Journal of Economic Perspectives*, 18(2):107–126, 2004.

[12] Pieter Wuille. BIP-32: Hierarchical deterministic wallets. Bitcoin Improvement Proposals, 2012. Available at https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki.